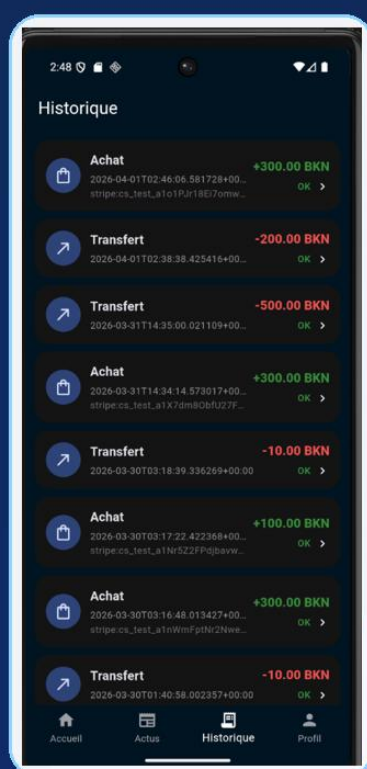
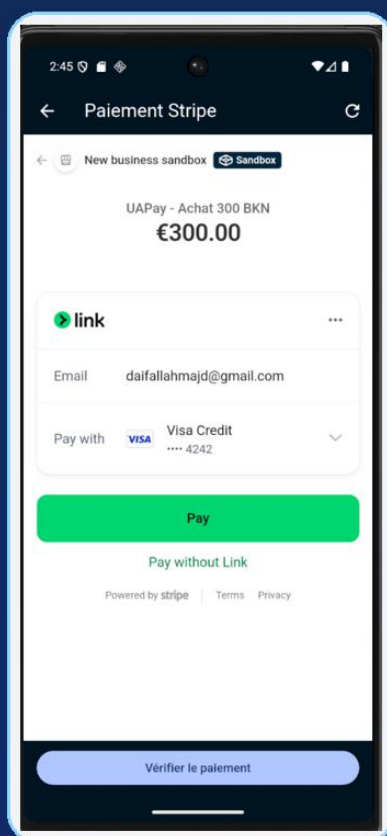
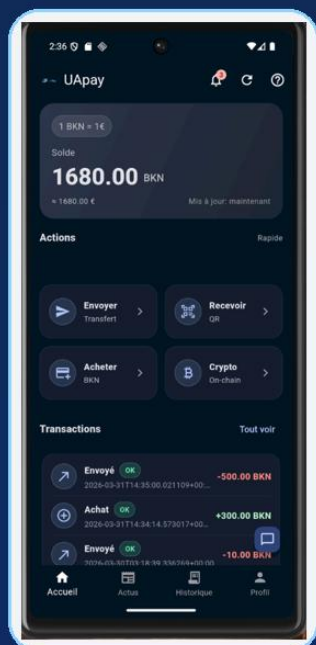


UAPay

Rapport technique et analyse d'infrastructure

Application mobile Flutter, backend Express, Supabase, Stripe et extension blockchain



Prototype full-stack orienté portefeuille numérique : achats BKN, transferts internes, QR, notifications, chatbot assisté et extension on-chain ERC-20.

Projet de programmation mobile

Auteur : Majd Daifallah

Date : 31 mars 2026

1. Synthèse d'inspection

Conclusion générale

Après inspection du dépôt, UAPay se présente comme un prototype full-stack crédible de portefeuille numérique. Le cœur du projet n'est pas seulement graphique : l'application met en œuvre une architecture réelle avec Flutter côté client, Supabase pour l'authentification et les données, un backend Node/Express pour Stripe et une extension blockchain ERC-20 déployable sur Base Sepolia.

Le point le plus convaincant du projet est la correction d'un problème fonctionnel concret : un paiement Stripe pouvait être validé sans crédit effectif du portefeuille. Le dépôt contient à la fois le diagnostic, les correctifs côté backend et mobile, ainsi qu'un script SQL de remise en état des données pour les environnements déjà déployés.

L'ensemble obtenu correspond bien à un projet académique avancé : il couvre les flux d'authentification, la gestion d'un solde BKN, les transferts internes, la réception par QR code, l'achat via Stripe, l'historique, les notifications, un chatbot capable de déclencher des transferts après confirmation, et une passerelle optionnelle vers des transferts on-chain.

2. Plan du rapport

- Présentation du projet et objectifs.
- Architecture technique et infrastructure d'exécution.
- Modèle de données, sécurité et fonctions SQL.
- Description détaillée des parcours utilisateur et des flux métier.
- Analyse du correctif paiement/wallet.
- Validation fonctionnelle illustrée par les captures d'écran.
- Limites actuelles et pistes d'amélioration.

3. Objectifs fonctionnels du projet

Le projet poursuit un objectif principal : fournir une application mobile de type wallet capable de gérer un actif applicatif nommé BKN. À partir du code et des captures validées, on peut identifier les objectifs suivants :

- permettre l'inscription, la connexion et la gestion du compte utilisateur ;
- attribuer automatiquement un portefeuille BKN et un solde initial ;
- autoriser les transferts entre utilisateurs par email, QR code ou commande conversationnelle ;
- intégrer un paiement Stripe afin d'acheter des BKN à un taux fixe de 1 BKN = 1 EUR ;
- conserver un historique des opérations et notifier l'utilisateur des événements importants ;
- préparer une extension blockchain capable d'émettre un transfert ERC-20 BKN sur un réseau EVM de test.

4. Architecture technique et infrastructure

L'architecture observée repose sur une séparation claire entre le client mobile, les services métiers, la couche de données et les services externes. Le dépôt racine est organisé autour de quatre blocs principaux :

Dossier	Rôle dans l'architecture
mobile/	Application Flutter : interface, navigation, flux utilisateur, appels Supabase et backend Stripe/EVM.
backend/	Serveur Node/Express : endpoints Stripe, vérification de statut, crédit du wallet et route de transfert ERC-20.
supabase/	Schéma SQL, RLS, trigger de création de compte, fonctions RPC et scripts correctifs.
contracts/	Contrat intelligent BKNToken.sol et sous-dossier Hardhat pour le déploiement sur Base Sepolia.
docs/	Notes de publication et documentation du correctif paiement/wallet.

Architecture technique de UAPay

Application mobile Flutter - Backend Express - Supabase - Stripe - Blockchain - IA locale

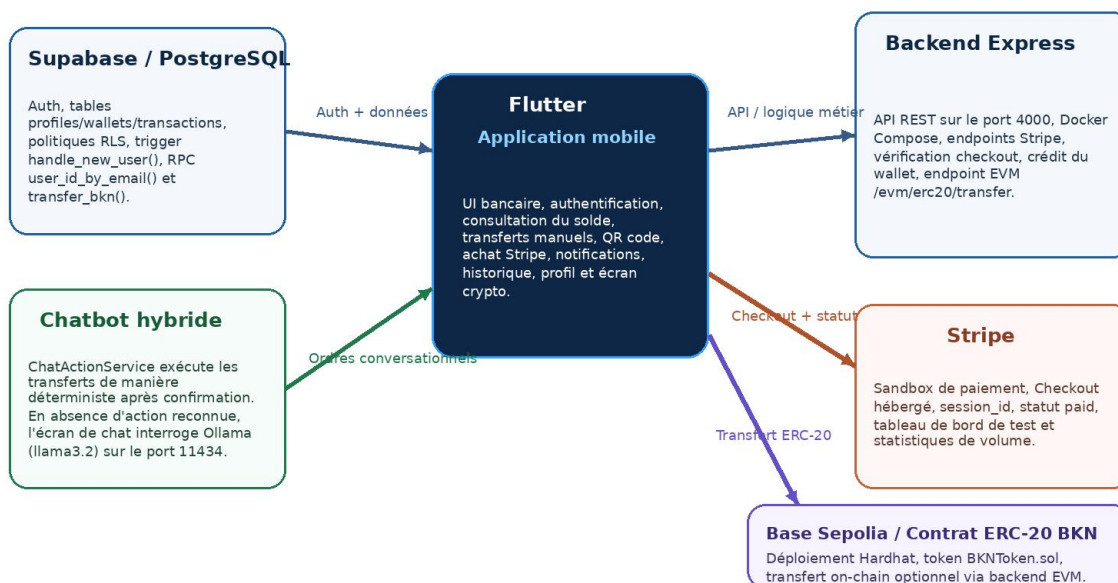


Figure 1 - Vue d'ensemble de l'architecture technique observée dans le projet UAPay.

4.1. Application mobile Flutter

Le client Flutter constitue le point d'entrée fonctionnel. Le fichier mobile/lib/main.dart initialise Supabase, active un système de notifications locales persistées, puis démarre l'interface avant le service de deep links afin d'éviter les écrans noirs au lancement. L'application adopte un thème sombre bancaire cohérent avec les captures fonctionnelles.

La navigation principale est assurée par AppShell, qui regroupe quatre onglets : Accueil, Actus, Historique et Profil. L'accueil expose les actions métier rapides (Envoyer, Recevoir, Acheter, Crypto) et un bouton flottant dédié au chatbot. Les écrans spécialisés couvrent également la vérification d'email, la réinitialisation du mot de passe, les paramètres de sécurité, les statistiques et le changement de compte.

4.2. Supabase : authentification, persistance et sécurité

Supabase joue un double rôle. D'une part, il fournit l'authentification utilisateur. D'autre part, il héberge la base PostgreSQL et la logique SQL associée au projet. La structure essentielle comprend les tables profiles, wallets et transactions. Les politiques RLS garantissent qu'un utilisateur ne lit et ne modifie que ses propres lignes.

Un trigger `handle_new_user()` initialise automatiquement profile et wallet lors de la création d'un compte. Deux fonctions RPC sont centrales : `user_id_by_email()` résout un destinataire à partir de son email, et `transfer_bkn()` exécute un transfert atomique avec verrouillage des lignes wallet, mise à jour des soldes et journalisation simultanée côté émetteur et récepteur.

4.3. Backend Node/Express et conteneurisation

Le backend est défini dans `backend/src/index.js` et exposé par Docker Compose sur le port 4000. Il applique Helmet, CORS, une limitation de débit globale, ainsi qu'une limitation renforcée pour le point d'entrée EVM. Le conteneur backend peut être démarré indépendamment de l'application Flutter, ce qui simplifie les tests locaux.

Endpoint	Méthode	Fonction
<code>/health</code>	GET	Vérification de disponibilité du backend.
<code>/create-checkout-session</code>	POST	Création d'une session Stripe Checkout à partir du montant BKN et du compte utilisateur.
<code>/checkout-status</code>	GET	Lecture du statut Stripe ; peut aussi déclencher le crédit du portefeuille lorsque <code>paid=true</code> .
<code>/stripe-webhook</code>	POST	Traitement du webhook <code>checkout.session.completed</code> et crédit idempotent du wallet.
<code>/success / /cancel</code>	GET	Pages de retour de paiement Stripe utilisées par le flux mobile.
<code>/evm/erc20/transfer</code>	POST	Relais de signature et d'envoi d'un transfert ERC-20 BKN en mode démo/testnet.

4.4. Services externes intégrés

Le projet consomme plusieurs services externes. Stripe fournit l'interface de paiement hébergée et les données de transaction du sandbox. Base Sepolia, via un RPC EVM configuré, sert de réseau de démonstration pour l'extension blockchain. Enfin, le chatbot peut déléguer les réponses libres à un service Ollama local (modèle `llama3.2`) accessible depuis l'émulateur Android sur le port 11434.

4.5. Déploiement blockchain

Le contrat `BKNToken.sol` s'appuie sur OpenZeppelin ERC20 et Ownable. Le sous-dossier `contracts/hardhat-deploy` prépare le déploiement sur Base Sepolia (chainId 84532) et l'opération de mint. L'écran Crypto de l'application reste volontairement orienté démonstration : l'interface prévient explicitement qu'il ne faut jamais saisir de vraie clé mainnet.

5. Modèle de données et mécanismes de sécurité

Modèle de données principal

Schéma observé dans supabase/schema.sql et extensions on-chain

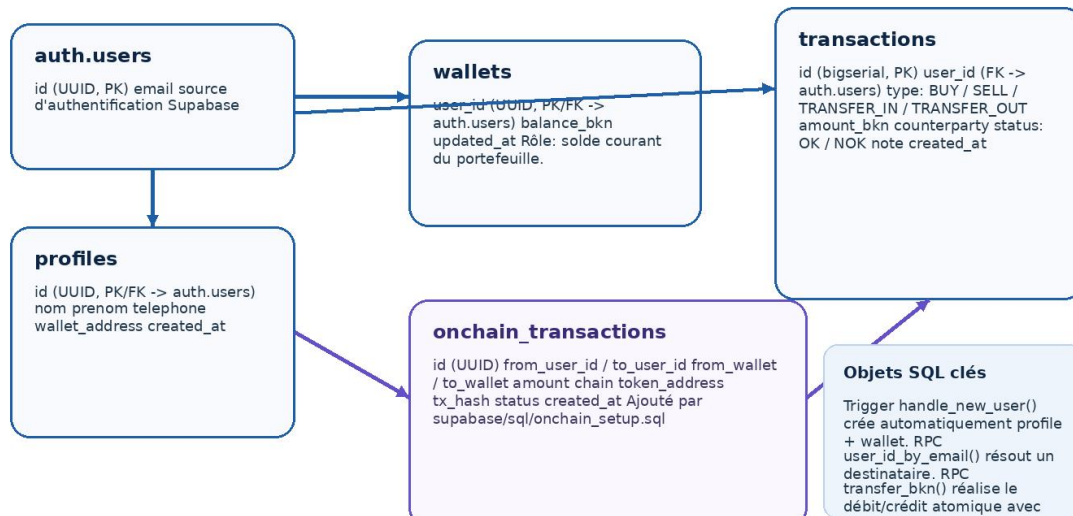


Figure 2 - Modèle de données principal déduit du schéma Supabase et des extensions on-chain.

5.1. Tables principales

Le schéma relationnel s'articule autour de trois tables de base et d'une table complémentaire pour l'extension on-chain :

Table	Description
profiles	Informations utilisateur : nom, prénom, téléphone, adresse wallet EVM éventuelle et date de création.
wallets	Solde BKN courant par utilisateur. La clé primaire est user_id, ce qui impose un portefeuille unique par compte.
transactions	Journal des achats, ventes simulées, transferts entrants et sortants. Les colonnes note et counterparty servent à la traçabilité.
onchain_transactions	Table ajoutée par le script onchain_setup.sql pour enregistrer les hashes de transactions blockchain.

5.2. Politiques RLS

Les politiques RLS sont activées sur profiles, wallets et transactions. Chaque table possède des règles explicites de select, insert et update associées à auth.uid(). Cette approche est importante dans un projet de wallet : le client mobile peut interagir directement avec Supabase tout en restant isolé au niveau des lignes.

5.3. Trigger et fonctions RPC

Le trigger on_auth_user_created appelle handle_new_user(), ce qui garantit la création d'un profile et d'un wallet avec solde initial. La fonction transfer_bkn() constitue la pièce centrale du transfert : elle refuse l'auto-transfert, vérifie le montant, s'assure de l'existence des wallets, verrouille les lignes, met à jour les soldes puis écrit deux transactions miroir.

5.4. Persistance locale côté mobile

Au-delà de Supabase, le client utilise SharedPreferences pour deux usages : la persistance des notifications et la mise en cache d'un historique minimal en mode offline. Le changement de compte, lui, s'appuie sur flutter_secure_storage afin de conserver de manière plus sûre les refresh tokens nécessaires au basculement entre sessions.

6. Parcours utilisateur validés par les captures

6.1. Authentification et création de compte

Le parcours d'accès repose sur deux écrans distincts. L'écran de connexion appelle `signInWithEmailAndPassword` via `SupabaseService`, puis redirige soit vers `VerifyEmailScreen` si l'email n'est pas confirmé, soit vers `AppShell` si l'authentification est complète. L'écran d'inscription collecte les informations minimales de profil avant l'appel `signUp()`.

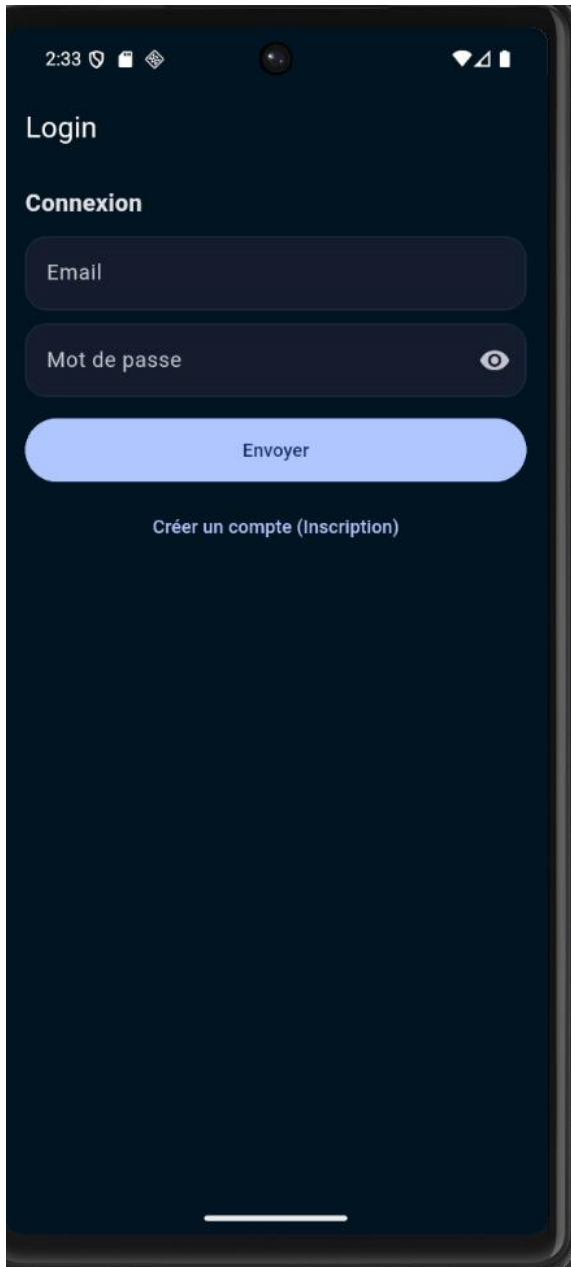


Figure 3 - Écran de connexion UAPay.

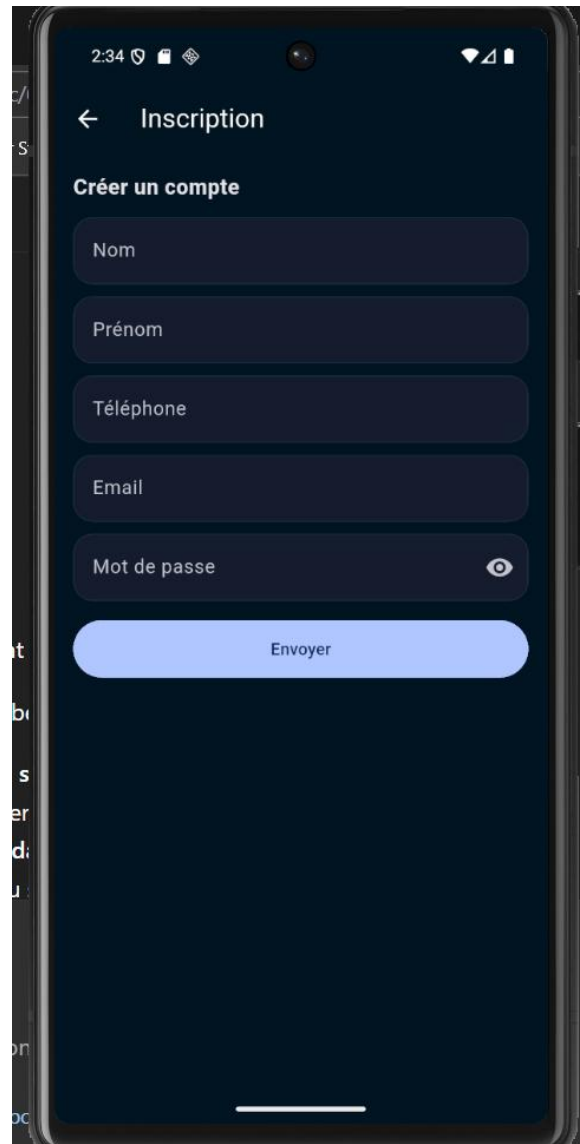


Figure 4 - Écran d'inscription d'un nouvel utilisateur.

6.2. Tableau de bord et lecture du solde

Une fois connecté, l'utilisateur accède à un tableau de bord qui synthétise le solde BKN, la conversion approximative en euros, les actions rapides et les dernières transactions. L'écran appelle `getBalance()` puis `getTransactions()` ; il synchronise aussi les notifications dérivées des nouveaux événements observés dans l'historique.

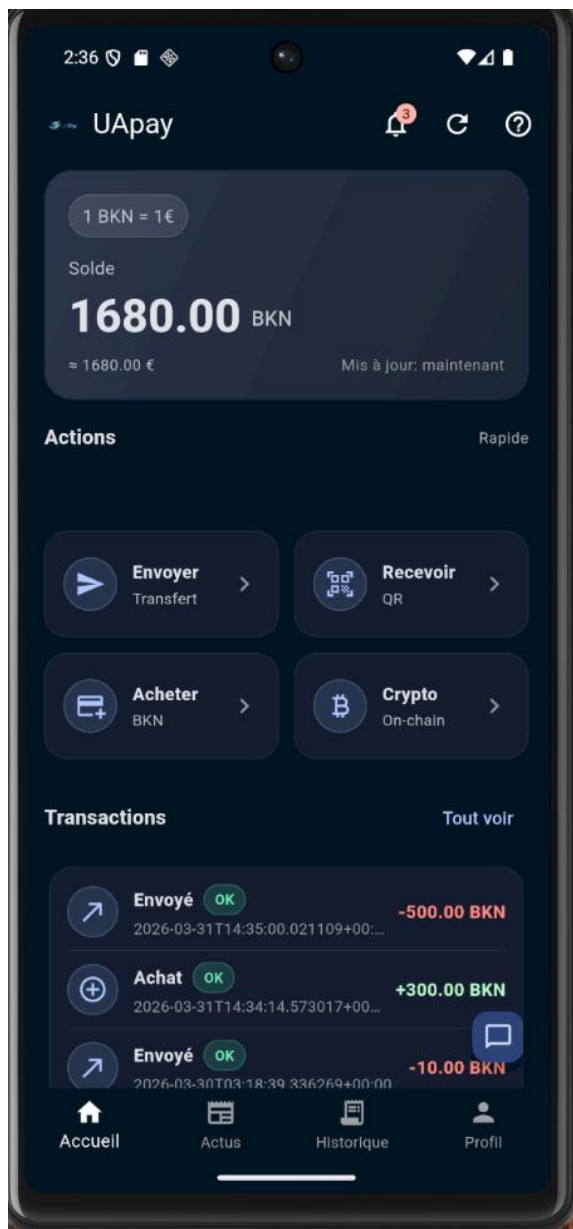


Figure 5 - Tableau de bord principal durant la validation fonctionnelle.

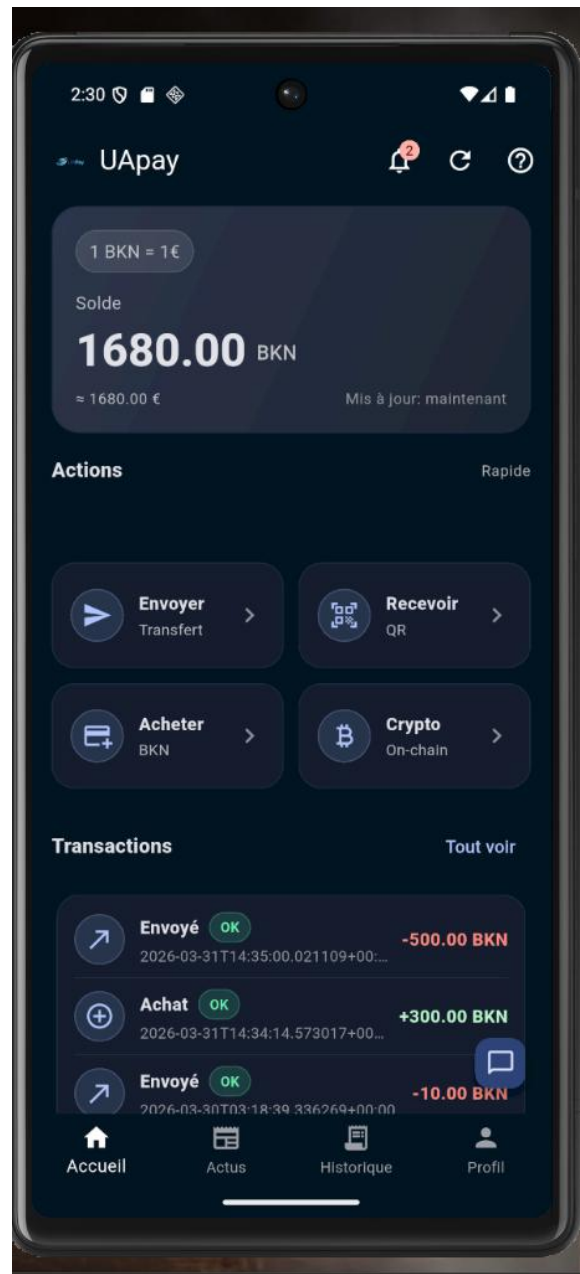


Figure 6 - Variante du tableau de bord avec compteur de notifications.

6.3. Transfert manuel et réception par QR code

Le transfert standard peut être déclenché par email ou en scannant un QR code. Dans le premier cas, l'application convertit l'email en user_id via la fonction RPC `user_id_by_email()`. Dans le second, le QR transporte un schéma `bkn://user/<uuid>` qui permet de préremplir directement le destinataire.

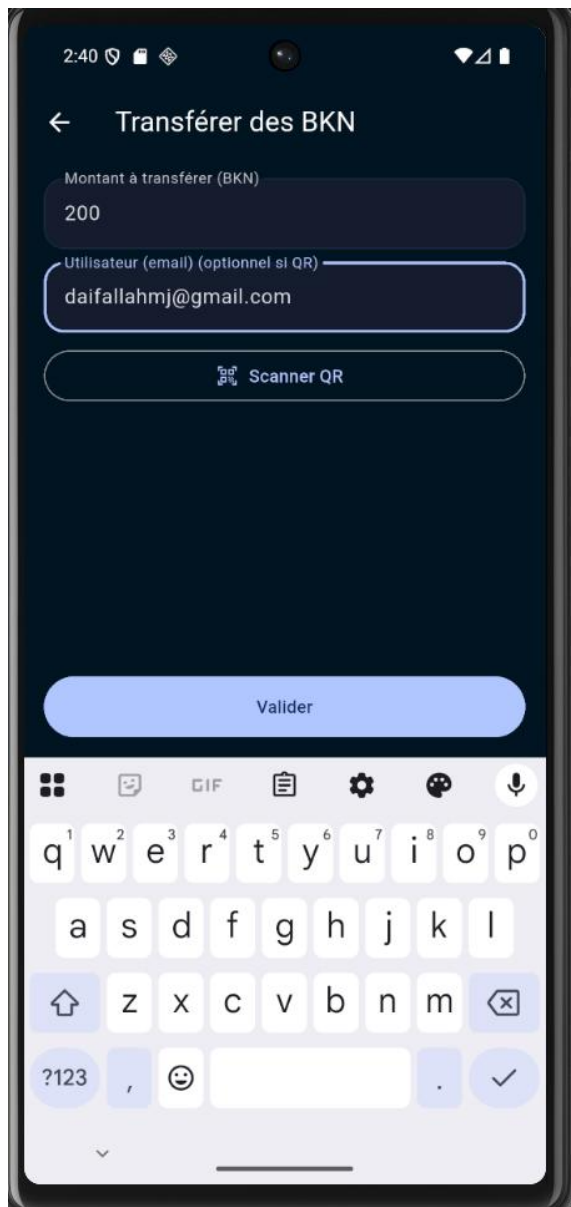


Figure 7 - Saisie d'un transfert manuel de 200 BKN.



Figure 8 - Écran de réception par QR code.

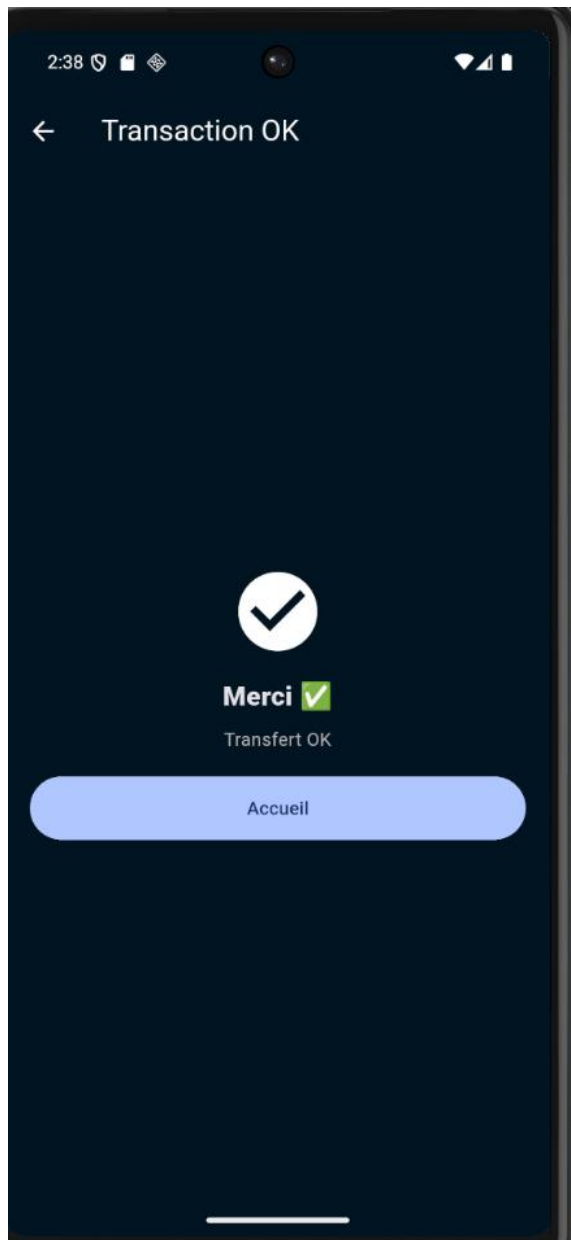


Figure 9 - Confirmation de réussite après exécution du transfert.



Figure 10 - Solde mis à jour côté expéditeur après l'envoi.

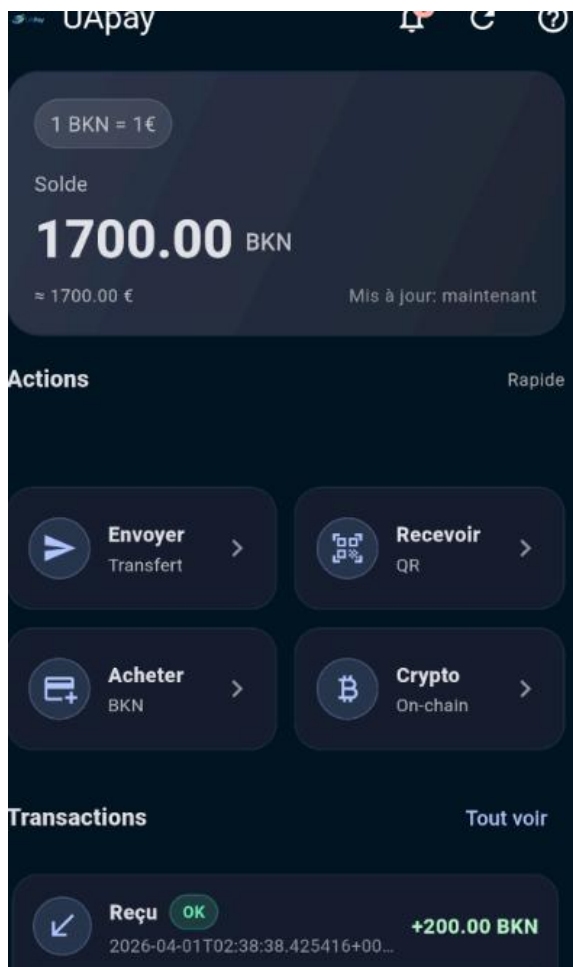


Figure 11 - Solde mis à jour côté destinataire après réception.

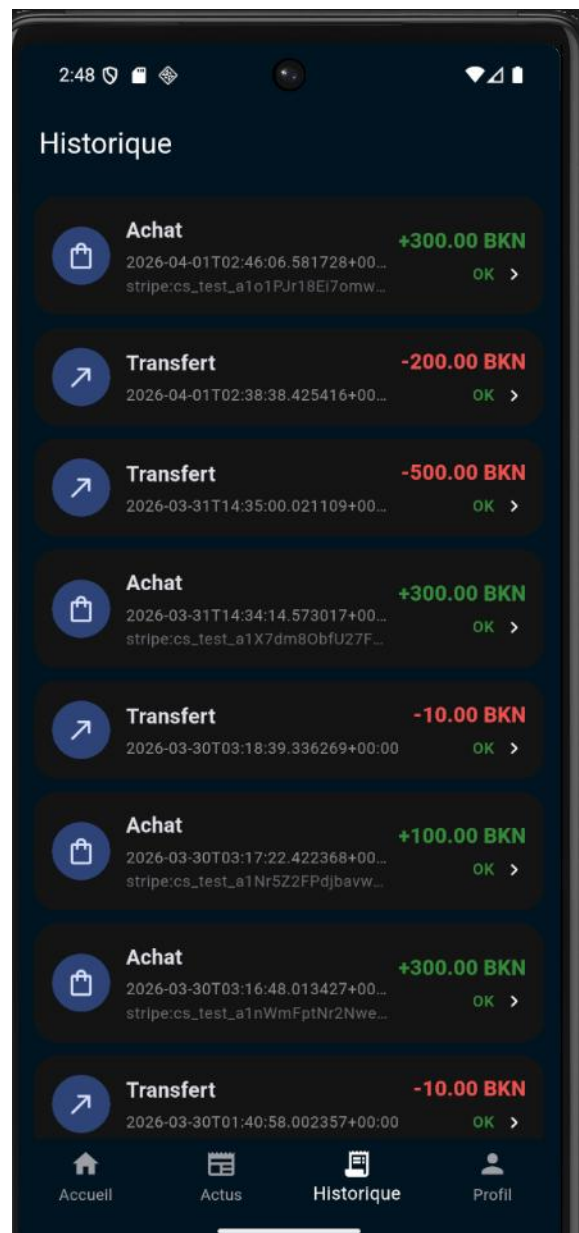


Figure 12 - Historique détaillé des achats et transferts.

6.4. Notifications applicatives

NotificationService alimente un centre de notifications persisté sur le terminal. Les achats confirmés, les paiements en attente, les fonds reçus et les échecs sont enregistrés puis affichés avec un indicateur d'état lu/non lu.



Figure 13 - Centre de notifications généré par les événements de l'application.

7. Flux de paiement Stripe et correctif du portefeuille

Le flux d'achat BKN est l'élément le plus important de l'infrastructure, car il combine mobile, backend Express, Stripe et base de données. L'écran Acheter des BKN crée une session Stripe, ouvre CheckoutWebViewScreen, puis interroge le backend sur /checkout-status afin de déterminer si le paiement est confirmé.

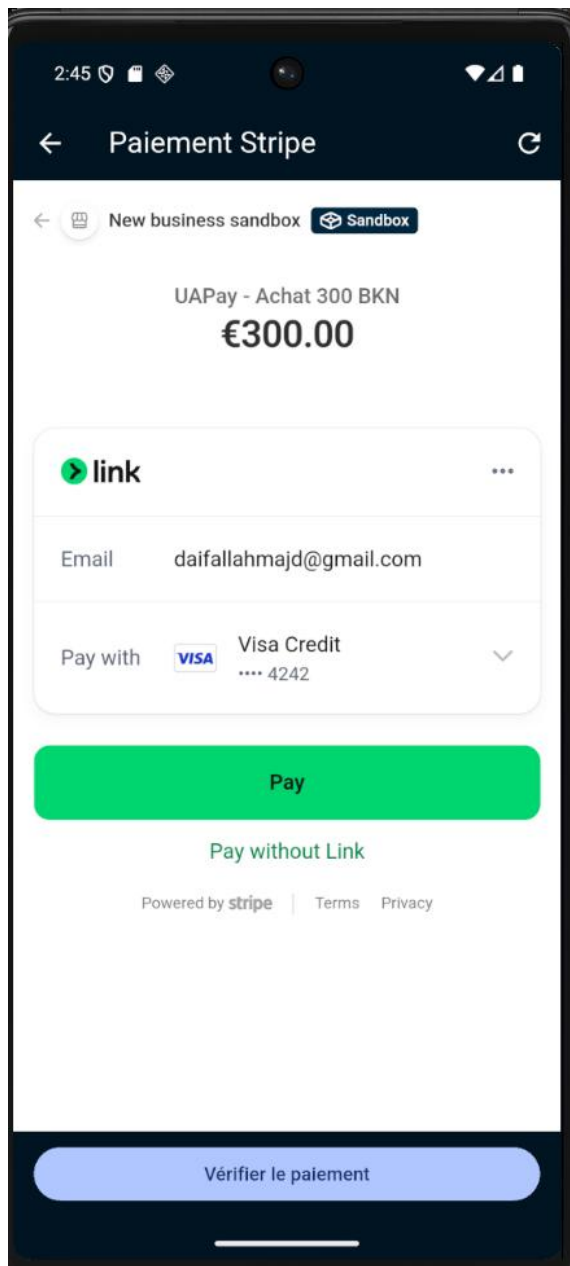


Figure 14 - Page Stripe Checkout intégrée dans l'application mobile.

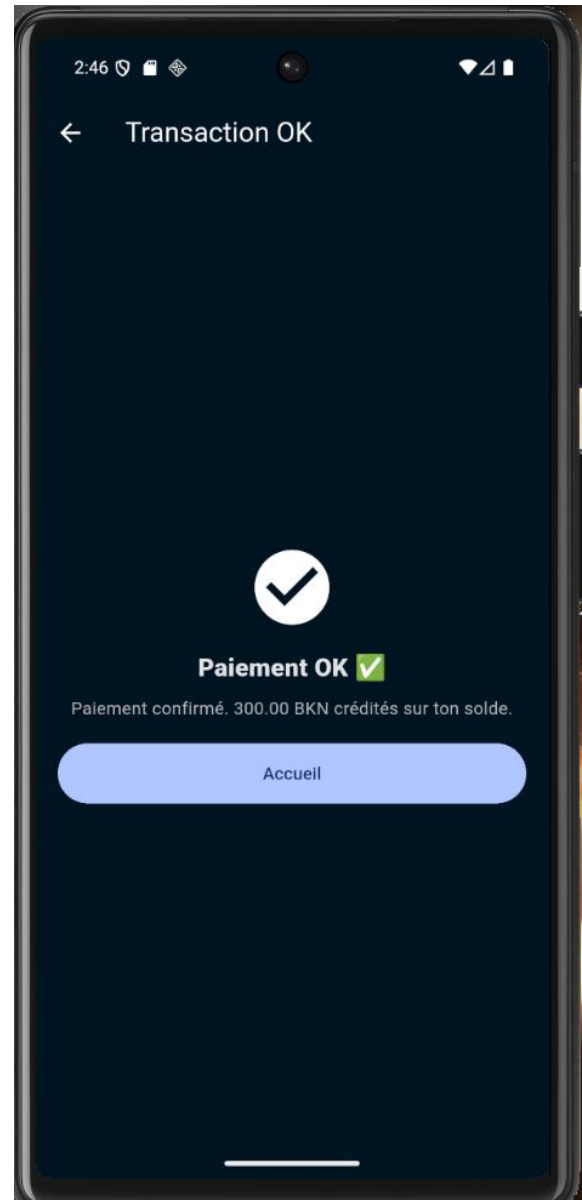


Figure 15 - Écran de confirmation du paiement réussi.



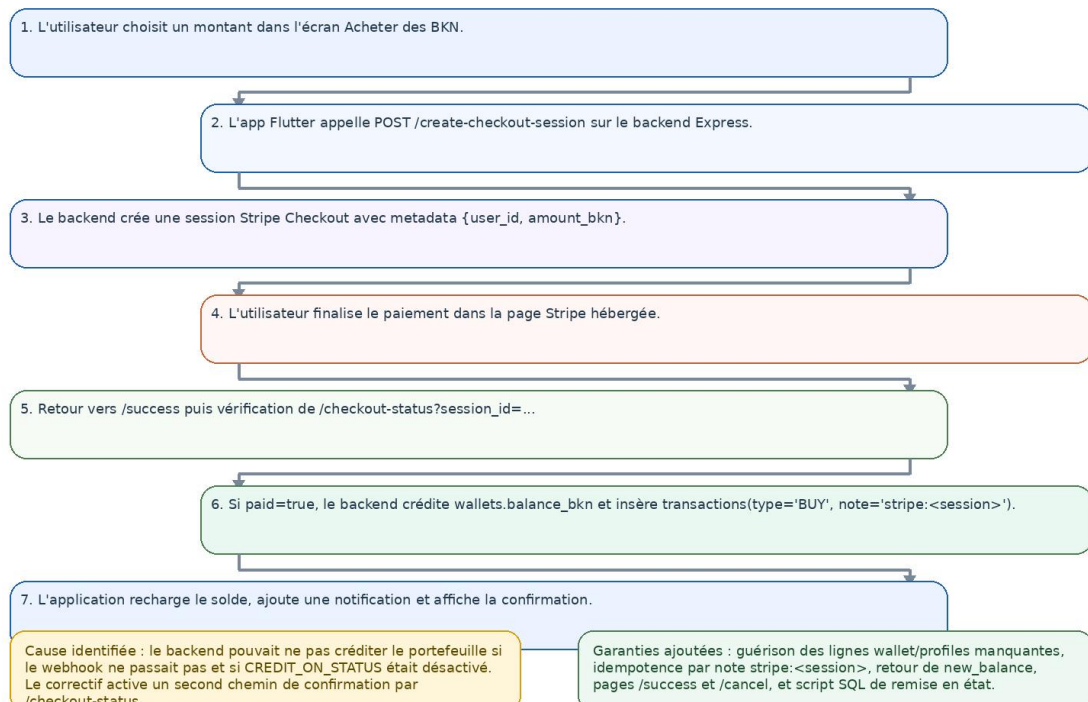
Figure 16 - Tableau de bord après crédit de +300 BKN.



Figure 17 - Tableau de bord Stripe sandbox montrant le paiement validé.

Flux de paiement Stripe et correctif appliqué

Le correctif rend le crédit du wallet robuste même si le webhook n'est pas reçu immédiatement.



7.1. Incident principal identifié

Les notes du dépôt indiquent clairement le dysfonctionnement observé : un paiement pouvait être accepté chez Stripe alors que le portefeuille restait inchangé. Plusieurs causes ont été documentées : dépendance excessive au webhook, création inconstante des lignes wallet, logique CORS peu robuste et URLs de retour de paiement fragiles.

7.2. Correctifs implémentés

- backend/src/index.js gère explicitement les origines avec un wildcard réel lorsque `ALLOWED_ORIGINS=*`.
- Le backend expose désormais des pages `/success` et `/cancel` cohérentes pour le retour depuis Stripe Checkout.
- Le paramètre `CREDIT_ON_STATUS` peut activer un crédit de secours lors de la vérification de `/checkout-status`.
- La fonction `creditStripeSessionIfNeeded()` vérifie l'idempotence via la note stripe:`<session_id>` avant tout crédit.
- SupabaseService recrée profile et wallet manquants lorsque nécessaire, y compris lors de la lecture du solde.
- Le script `supabase/sql/live_fix_payment_and_wallets.sql` remet en état les tables, le trigger, les politiques RLS et les fonctions RPC.

7.3. Conséquence fonctionnelle

Le résultat visible dans les captures est cohérent avec la logique corrigée : la transaction d'achat apparaît dans l'historique, la notification d'achat confirmé est générée, et le solde de l'utilisateur augmente effectivement de 300 BKN. Le correctif renforce donc la fiabilité du parcours de recharge du portefeuille.

8. Chatbot assisté et extension on-chain

8.1. Transfert déclenché par chatbot

L'écran ChatBot n'est pas un simple démonstrateur textuel. ChatActionService détecte les formulations de type « envoyer/payer/transférer », extrait un montant et un destinataire, résout le bénéficiaire par email ou par nom, puis demande explicitement une confirmation. En cas de réponse positive, il appelle transferToUserId() et exécute le même flux métier que le transfert manuel.

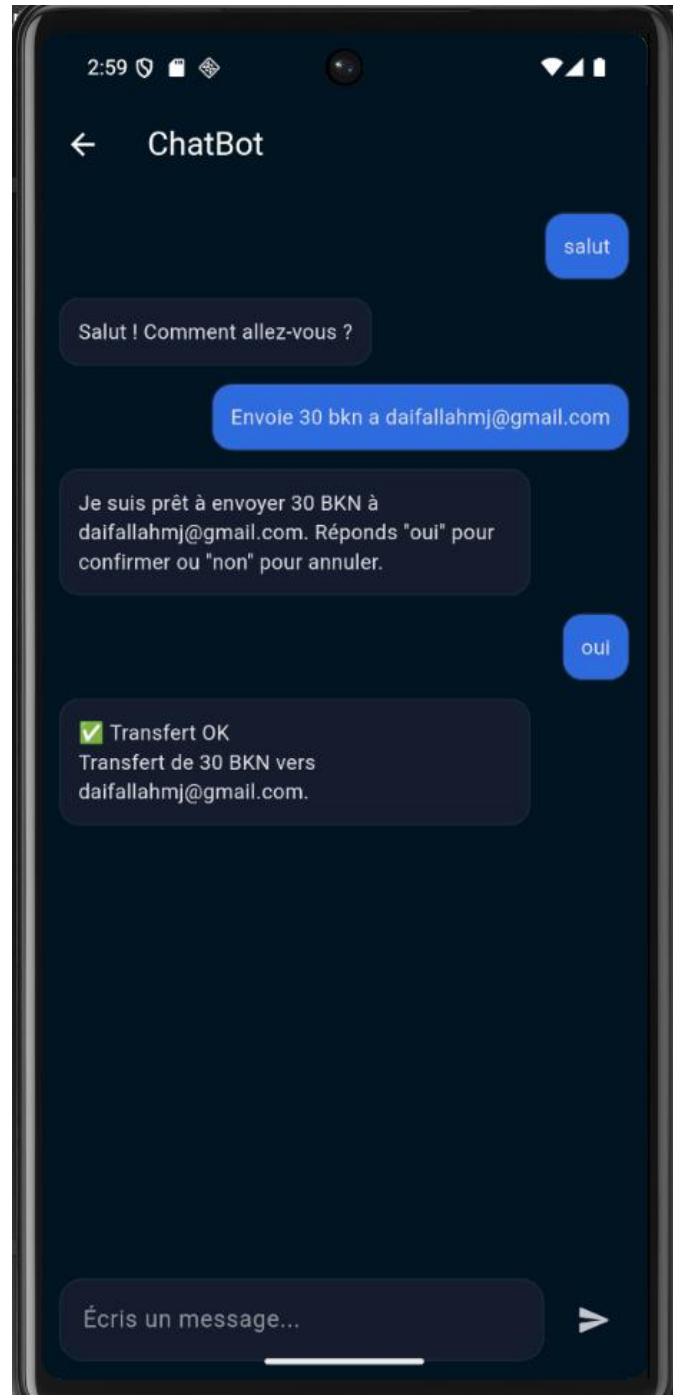


Figure 19 - Conversation de validation d'un transfert de 30 BKN via le chatbot.

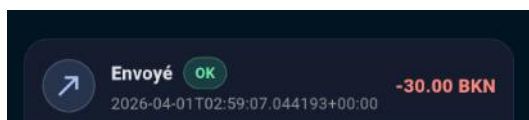


Figure 20 - Trace du débit de 30 BKN côté expéditeur après

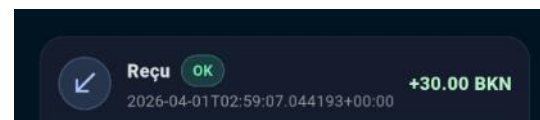


Figure 21 - Trace du crédit de 30 BKN côté destinataire

Cette fonctionnalité apporte une richesse supplémentaire au projet : le chatbot n'exécute pas de logique sensible par génération de texte libre, mais par une couche d'actions déterministes qui sécurise la transaction avant d'autoriser le modèle local à répondre aux demandes non structurées.

8.2. Extension blockchain ERC-20

L'écran Crypto matérialise l'ouverture du projet vers un registre externe. La configuration attend un RPC EVM, un identifiant de chaîne, une adresse de contrat BKN et une URL de backend EVM. Le service BlockchainService appelle alors /evm/erc20/transfer, qui utilise ethers.js côté serveur pour construire et diffuser la transaction.



Figure 22 - Écran de transfert on-chain en mode de configuration incomplète.

Architecture technique de UAPay

Application mobile Flutter - Backend Express - Supabase - Stripe - Blockchain - IA locale

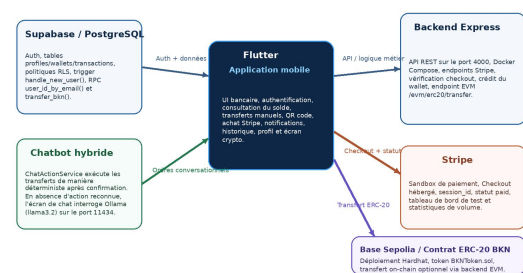


Figure 23 - Positionnement de l'extension blockchain dans l'architecture globale.

8.3. Important sur le périmètre

L'extension on-chain doit être présentée comme une capacité expérimentale de démonstration, et non comme une brique de production finalisée. Le code et les écrans signalent d'ailleurs eux-mêmes ce caractère de démo : l'utilisateur est averti qu'il ne faut jamais saisir de vraie clé privée de production.

9. Validation technique et captures d'infrastructure

Au-delà des écrans mobiles, plusieurs captures techniques permettent de vérifier l'environnement de test et l'infrastructure de support du projet.

```

[+] Userdata4/Downloads/UPAY/UPAY_payment_patch/waypay_fixed/docker compose up --build
[+] Building 2.8s (12/12) FIMISHED
[+] Building image local base definitions 0.1s
[+] > reading from stdin 0.00s
[+] Internal build definition from Dockerfile 0.1s
[+] > Transferring Dockerfile: 136B 0.1s
[+] Internal build metadata for docker.io/library/node:20-alpine 0.0s
[+] Internal build dockerignore 0.0s
[+] > Transferring content: 2B 0.0s
[+] Internal build metadata for docker.io/node:20-alpine:sha256-f9983782520225e6a6b6f9f3566c1f08e55373d6ed4d15313080 0.0s
[+] Internal build build context 0.0s
[+] > Transferring content: 99B 0.0s
[+] CACHED [2/3] WORKDIR /app 0.0s
[+] CACHED [3/4] COPY package.json package-lock.json* 0.0s
[+] CACHED [4/5] RUN npm run install --omit-dev 0.0s
[+] CACHED [5/5] COPY src ./src 0.0s
[+] exporting image 0.0s
[+] exporting layers 0.0s
[+] writing image sha256:4c6e2799113a24096201f0d6d5126181b049546d06be2b25884e24c29a62 0.0s
[+] naming to docker.io/library/waypay-backend 0.0s
[+] resolving provenance for metadata file 0.0s
[+] done 0.0s
[+] waypay-backend Built 0.0s
[+] vContainer waypay-backend-1 Created 1.3s
Attaching to waypay-backend-1
backend-1
backend-1 | waypay-stripe-backend@0.0.0 start
backend-1 | node src/index.js

```

Figure 24 - Démarrage du backend Node/Express via Docker Compose.



Figure 25 - Conteneur backend visible dans Docker Desktop.

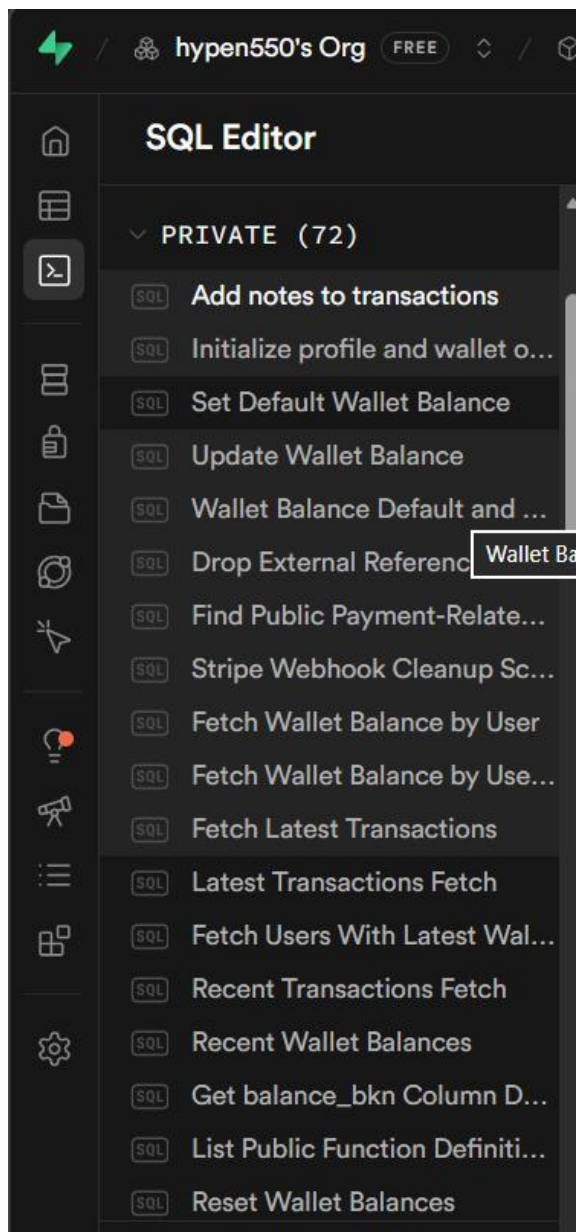


Figure 26 - Scripts SQL présents dans l'éditeur Supabase.

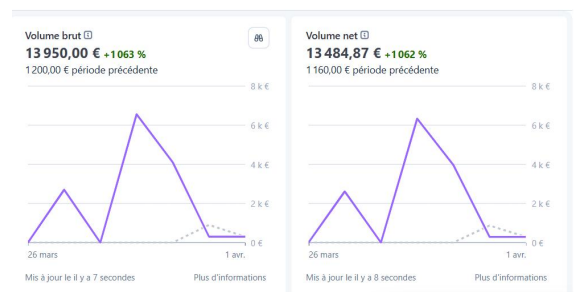


Figure 27 - Statistiques de volume visibles dans le dashboard Stripe sandbox.

Ces éléments confirment que le projet n'est pas seulement décrit théoriquement : il a été exécuté, connecté à Stripe en sandbox, exposé par un backend Dockerisé et rattaché à un projet Supabase réel.

9.1. Lecture infrastructurelle

Sur le plan de l'infrastructure, UAPay combine donc des composants locaux (émulateur Flutter, backend Express, éventuel Ollama local) et des services managés externes (Supabase, Stripe, réseau EVM de test). Cette composition est particulièrement adaptée à un projet académique : elle permet d'illustrer une vraie architecture distribuée sans devoir administrer soi-même une base PostgreSQL, un orchestrateur ou une passerelle de paiement.

- Flux métier central robuste : transfert BKN et achat BKN réellement connectés à une logique de données.
- Sécurisation des données par RLS, trigger d'initialisation et logique SQL transactionnelle.
- Présence d'un correctif documenté pour un incident concret de paiement non répercuté dans le portefeuille.
- Validation fonctionnelle riche grâce aux captures d'écran et à la traçabilité dans l'historique.
- Ouverture vers des extensions avancées : chatbot orienté action, changement de compte, cache offline et blockchain.

10. Difficultés rencontrées

10.1. Synchronisation entre paiement Stripe et crédit du wallet

La difficulté technique majeure du projet a consisté à rendre parfaitement cohérent un flux distribué entre quatre couches : l'application Flutter, le backend Express, Stripe Checkout et Supabase. En pratique, un paiement pouvait être validé côté Stripe sans que le solde BKN n'apparaisse immédiatement crédité dans le wallet. Ce décalage venait du fait que la confirmation métier ne dépendait pas d'un seul composant : elle impliquait le retour de session Checkout, la lecture de l'état de paiement, la présence effective d'une ligne wallet en base, puis l'écriture atomique de la transaction et du nouveau solde.

Cette difficulté a obligé à traiter plusieurs cas de défaillance en même temps : redirections Stripe incomplètes, dépendance au statut de session, risque de double crédit, wallet absent pour certains comptes, et nécessité de refléter immédiatement le résultat dans l'interface mobile. La réponse apportée dans le projet repose sur un crédit idempotent, la reconstruction des enregistrements manquants côté Supabase et une logique de vérification explicite via /checkout-status. Autrement dit, la vraie complexité n'était pas d'afficher un écran de paiement, mais d'assurer la cohérence finale entre preuve de paiement, écriture en base et solde visible par l'utilisateur.

10.2. Coordination d'une infrastructure multi-services

Une seconde difficulté importante a concerné la coordination de l'environnement d'exécution. Le projet assemble en effet des composants hétérogènes : application mobile Flutter, base Supabase distante, backend Node.js conteneurisé avec Docker, et services Stripe en sandbox. Dans un tel contexte, un simple défaut de configuration peut bloquer tout le parcours d'achat : mauvaise origine CORS, URL backend incorrecte selon l'émulateur ou le navigateur, variable d'environnement absente, clé Stripe mal injectée, ou divergence entre les identifiants attendus par le mobile et ceux disponibles côté backend.

Le projet montre précisément cette difficulté d'intégration. Pour qu'un achat fonctionne de bout en bout, il faut que les valeurs de configuration restent compatibles entre plusieurs environnements locaux ou de test, que le backend soit joignable, que Stripe puisse rediriger correctement l'utilisateur, et que Supabase accepte ensuite les écritures métier. La difficulté rencontrée ne se résume donc pas à un bug isolé : elle tient au caractère distribué de l'architecture, où chaque maillon doit être correctement configuré pour que le flux complet soit validé.

10.3. Validation fonctionnelle, traçabilité et limites de démonstration

Enfin, une difficulté plus méthodologique a porté sur la validation du projet. Une application de wallet ne peut pas être évaluée uniquement sur son interface : il faut prouver que chaque action produit réellement un effet mesurable. Cela implique de croiser plusieurs indices : variation du solde avant/après opération, apparition de la transaction dans l'historique, notification locale, confirmation Stripe, et, lorsque nécessaire, lecture des journaux backend ou des tables Supabase. La collecte de captures cohérentes pour démontrer le bon fonctionnement constitue donc elle-même une difficulté du projet.

Cette exigence de preuve apparaît aussi dans l'extension blockchain. L'écran crypto est présent et l'infrastructure ERC-20 existe, mais son usage reste volontairement cantonné à un réseau de démonstration. La difficulté ici est double : montrer l'ouverture technique vers un transfert on-chain tout en assumant honnêtement les limites de sécurité et de configuration d'une telle fonctionnalité. Le rapport doit donc présenter cette partie comme une extension expérimentale validée en test, et non comme une fonctionnalité de production déjà industrialisée.

11. Analyse critique

11.1. Points forts observés

- Architecture modulaire compréhensible et réaliste pour un projet mobile full-stack.

- Flux métier central robuste : transfert BKN et achat BKN réellement connectés à une logique de données.
- Sécurisation des données par RLS, trigger d'initialisation et logique SQL transactionnelle.
- Présence d'un correctif documenté pour un incident concret de paiement non répercuté dans le portefeuille.
- Validation fonctionnelle riche grâce aux captures d'écran et à la traçabilité dans l'historique.
- Ouverture vers des extensions avancées : chatbot orienté action, changement de compte, cache offline et blockchain.

11.3. Améliorations recommandées

Axe	Amélioration proposée
Sécurité	Externaliser strictement toutes les clés sensibles et éviter toute saisie de clé privée côté client final.
Backend	Séparer routes, contrôleurs, services Stripe et services EVM pour améliorer la maintenabilité.
Tests	Ajouter des tests Flutter sur les écrans critiques et des tests backend sur les endpoints de paiement.

12. Conclusion

L'inspection du projet UAPay montre un prototype full-stack abouti pour un contexte académique. Cette application relie de manière cohérente un client Flutter, une base Supabase protégée par RLS, un backend Express conteneurisé, un paiement Stripe de test et une extension blockchain ERC-20. Le point le plus marquant reste la correction du flux de paiement, qui rend le crédit du portefeuille beaucoup plus fiable grâce à l'idempotence, à la réparation automatique des wallets et à la vérification de statut côté backend.

Annexe A. Détail des paiements Stripe sandbox

Cette capture complète la validation du flux de paiement en montrant la liste détaillée des paiements associés, leurs montants, leurs statuts de réussite, le type de source utilisé et la date d'exécution dans le dashboard Stripe sandbox.

N

New business sandbox

ua pay

▼

Accueil

Soldes

Transactions

Clients

Catalogue de produits

Raccourcis

Analyse des paiements

Recouvrement de rev...

Facturation à l'usage

Factures

Présentation de Billing

Produits

Payments

Billing

Reporting

Plus

Rechercher

Paie

ments

asso

ciés

RISQUE	MONTANT		TYPE DE SOURCE	CLIENT	APPAREIL	ADRESSE IP DE L'APPAREIL	DATE
7	300,00 €	Réussi ✓	4242		sdk_gphone64_x86_64	194.199.98.174	1 avr. à 13:24
63	300,00 €	Réussi ✓	4242		sdk_gphone64_x86_64	194.199.98.174	1 avr. à 13:12
15	300,00 €	Réussi ✓	4242		sdk_gphone64_x86_64	194.199.98.174	1 avr. à 13:00
37	300,00 €	Réussi ✓	Link		sdk_gphone64_x86_64	194.199.98.174	31 mars à 14:34
7	100,00 €	Réussi ✓	Link		sdk_gphone64_x86_64	92.49.116.7	30 mars à 03:17
27	300,00 €	Réussi ✓	Link		sdk_gphone64_x86_64	92.49.116.7	30 mars à 03:16
61	300,00 €	Réussi ✓	Link		sdk_gphone64_x86_64	92.49.116.7	30 mars à 02:40
21	300,00 €	Réussi ✓	Link		sdk_gphone64_x86_64	92.49.116.7	30 mars à 02:38
46	300,00 €	Réussi ✓	Link		sdk_gphone64_x86_64	92.49.116.7	30 mars à 02:38

Affichage de 1 à 10 sur environ 50 éléments

Préc. Suiv.

Figure A1 - Liste détaillée des paiements associés et validés dans Stripe sandbox.